CHAPTER 1 · INTRODUCTION

Page No. :
Date : / /

* Software :-

Software is

- instructions <computer programs> that when executed provide desired features, functions and performance.
- data structures that enable the programs to adequately manipulate information
- descriptive information in both hard copy and virtual forms that describe the operation and use of the programs.

* Characteristics Of Software :-

(i) Software is developed or engineered ; it is not manufactured in the classical sense.

- In both the activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems that are non existent for software
- Both activities are dependent on people, but the relationship between people applied & work accomplished is different.
- Software costs are concentrated in engineering. This means that software projects cannot be managed as if they were manufacturing projects.

(ii) Software doesn't wear out.

- Fig 1. depicts failure rate as a function of time for hardware. The relationship is called "bathtub curve", indicates hardware exhibits relatively high failure rates early in its lifetime ; defects are corrected & the failure rate drops to a steady state - level for some time & as time passes, failure rate rises again as hardware components suffer from cumulative effects of dust, vibration & other environmental maladies.

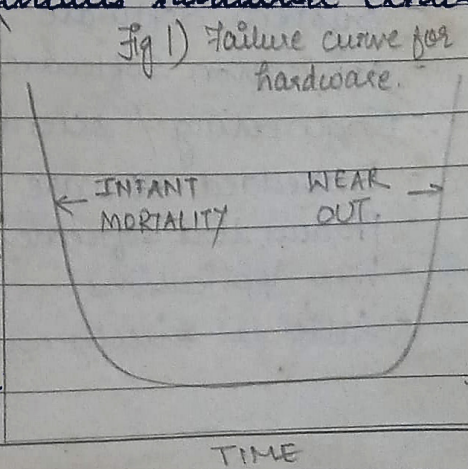
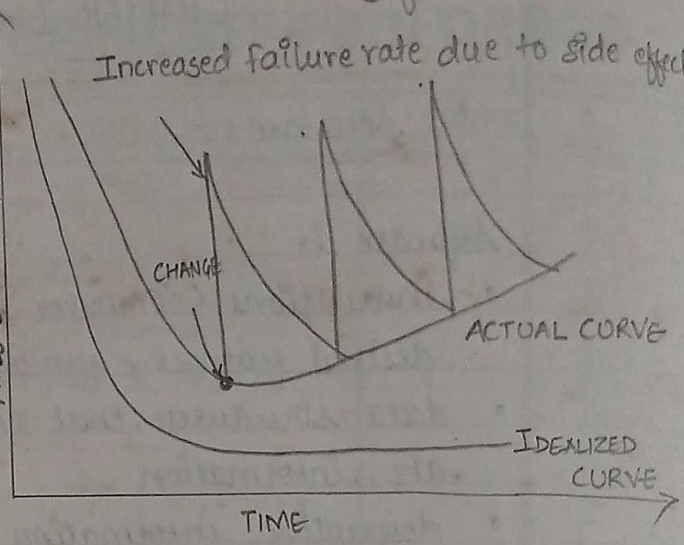


Fig 1) Failure curve for hardware.

Fig (2). Failure curve for software



- Software is not susceptible to the environmental maladies that cause hardware to wear out. Therefore, the failure rate curve for sfw should take the form of "idealized curve". Undiscovered defects will cause high failure rates early in the life of a program. However, these are corrected & the curve flattens. The idealized curve is a gross oversimplification of actual failure models for sfw. Hence, software doesn't wear out it deteriates.

(iii) Although the industry is moving toward component-based construction, most software continues to be custom built.

- In hardware world, component reuse is a natural part of engineering process. In software world, it is something that has only begun. Modern reusable components encapsulate both data & processing that is applied to the data, enabling engineers to create new applications from reusable parts. &
- For eg, today's interactive user interfaces are built with reusable components that enable creation of graphics, pull down menus, a variety of interaction mechanisms. The data structures & processing details required to build interface are contained within library of reusable components of interface construction.

* Software Application Domains :-

- System Software → OS, device drivers, firmware, Pgming Lang Translators, Utility
- Application Software → word processor, database sfw, mms, educational, web browsers
- Engineering / scientific software
- Embedded software
- Product line software
- Web Applications
- Artificial Intelligence software

Page No. :
Date : / /

* Software Engineering

- The application of a systematic, disciplined, quantifiable approach to the development, operation & maintenance of software; that is the application of engineering to software.
- The study of approaches as in (1).

- Software engineering is a layered technology. Fig 3. below shows that any engineering approach must rest on an organizational commitment to quality.

- Total Quality management, Six sigma and similar philosophies foster a continuous process improvement culture, & this culture ultimately leads to development of increasingly more effective approaches to software engineering. The bedrock of software engineering is quality focus.

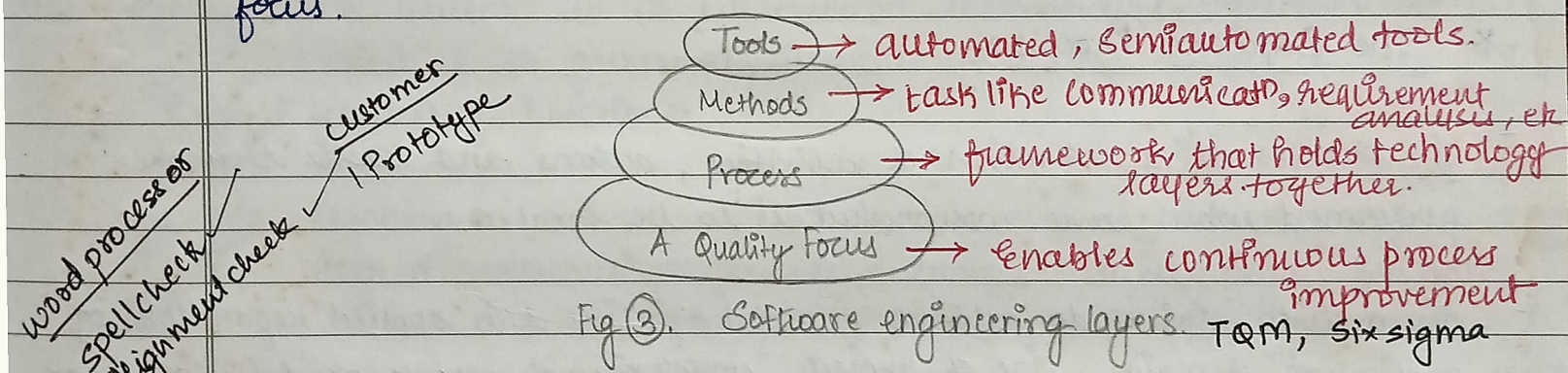


Fig (3). Software engineering layers.

- The foundation for software engineering is the process layer. The software engineering process is the glue that holds the technology layers together & enables rational & timely development of computer software.
- Process defines a framework that must be established for effective delivery of software engineering technology.
- The software process forms the basis for management control of sfw projects & establishes context in which technical methods are applied, work products are produced, milestones are established, quality is ensured & change is properly managed.

- Software engineering methods provide technical how-to's for building software. Methods encompass a broad array of tasks that include communication, requirements analysis, design modelling, program construction, testing & support.

- Software engineering methods rely on a set of basic principles that govern each area of technology & include modelling activities & other descriptive techniques.

- Software engineering tools provide automated or semiautomated support for the process & methods. When tools are integrated so that information created by one tool can be used by another, a system for the support of software development; called computer-aided software engineering is established.

* The Software Process :-

- A process is a collection of activities, actions and tasks that are performed when some workproduct is to be created.

- An activity strives to achieve a broad objective & is applied regardless of application domain, size of project, complexity of effort or degree of rigor with which software engineering is to be applied.

- An action encompasses a set of tasks that produce a major work product
- A task focusses on a small, but well-defined objective that produce a tangible outcome.

- A process is not a ~~tangible~~ rigid prescription for how to build computer software. Rather, it is an adaptive approach that enables people doing work to pick & choose appropriate set of work actions & task.

- The intent is always to deliver software in a timely manner & with sufficient quality to satisfy those who have sponsored its creation & those who

will use it.

- A process framework establishes foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of its size & complexity.
- In addition, the process framework encompasses set of umbrella activities applicable across entire software process.
- A generic process framework for software engineering encompasses following five activities :—

activity — Communication
action → with customer
→ with developer
→ with colleague
task → telephonic communication
• through emails
• through feedback.

(i) Communication :-

- Before any technical work can commence, it is critically important to communicate & collaborate with customer and other stakeholders.
- The intent is to understand stakeholder's objectives for the project & to gather requirements that help define software features & functions.

• through reviews or deliverables

(ii) Planning :-

- ~~Any~~ A software project is a complicated journey, planning activity creates a "map" that helps guide the team as it makes the journey.
- Software project plan - defines the software engineering work by describing technical tasks to be conducted, risks that are likely, resources that will be required, workproducts to be produced & a work.

(iii) Modelling :-

- A software engineer creates models to better understand software requirements & design that will achieve requirements of a software.

(iv) Construction :-

- This activity combines code generation & testing that is required

to uncover errors in the code.

(v) Deployment :-

- The software (as complete entity or partially completed increment) is delivered to the customer who evaluates the delivered product & provides feedback based on evaluation.

- These process framework activities may be used iteratively as a project progresses.

- Software engineering process framework activities are complemented by a number of umbrella activities.

- These activities applied throughout the software process help software team manage & control progress, quality, change & risk.

* Typical umbrella activities include:-

- Software Project Tracking and control :-
  allows software team to assess progress against project plan & take necessary actions to maintain schedule.

- Risk management :-
  assesses risks that may affect the outcome of project or quality of product

- Software Quality assurance :-
  defines & conducts the activities required to ensure software quality

- Technical Reviews :-
  assesses software engineering work products in an effort to uncover & remove errors before they propogate to next activity

- Measurements :-
  defines & collects process, project & product measures that assist the team in delivering software that meets stakeholders needs.

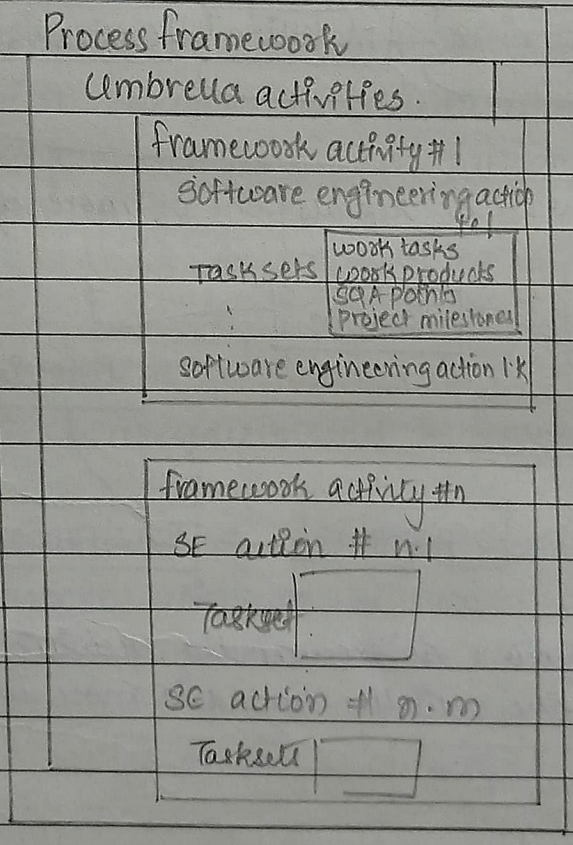
Page No. :
Date : / /

- **Software Configuration management :-**
manages the effects of change throughout the software process.

- **Reusability management :-**
defines criteria for work product reuse & establishes mechanisms to achieve reusable components.

- **Work product preparation & production :-**
encompasses activities required to create work products such as models, documents, logs, forms & lists.

* **Generic Process Model :**



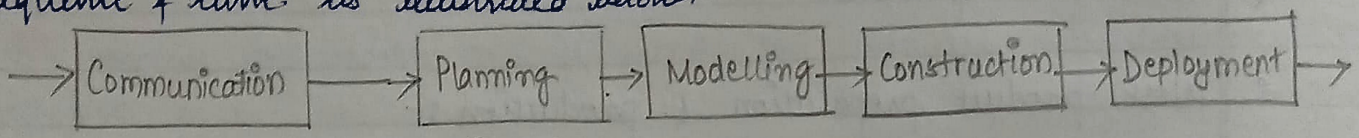
Framework.
activity -
① Requirement gathering
action 1. Mobile communication
Taskset - logs of meetings
- final streamlined requirements
milestone
- req. met ✓
SQA points
- req. with quality

- Software process as depicted in Fig. 4 above, each framework activity is populated by set of software engineering actions.

- Each software engineering action is defined by task set that identifies work tasks that are to be completed, workproducts

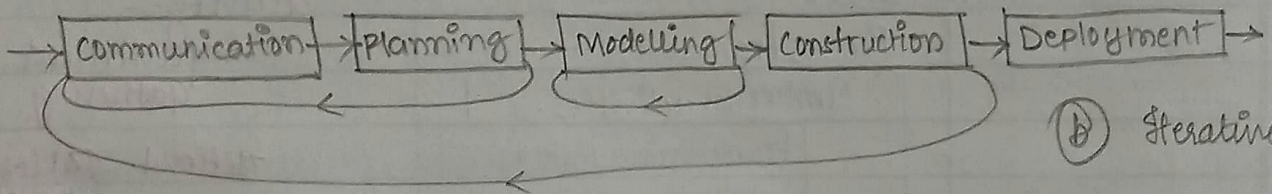
that will be produced, quality assurance points that will be required and milestones that will be used to indicate progress.

- A generic process framework has the five framework activities & umbrella activities.
- The process flow describes how framework activities & actions & tasks that occur within each framework activity are organized with respect to sequence & time as illustrated below:-



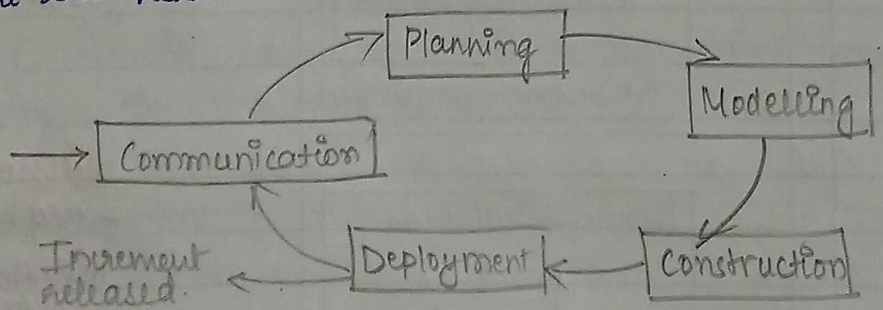
(a) Linear process flow.

- Linear Process Flow :- Linear process flow executes each of the five framework activities in sequence, beginning with communication & culminating with deployment.



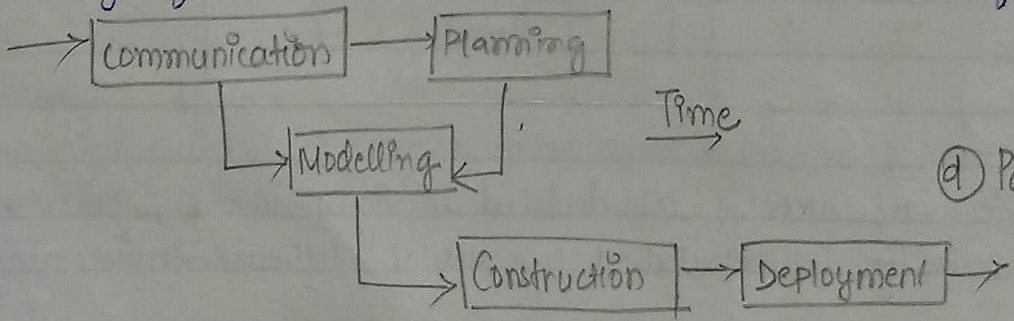
(b) Iterative process flow.

- Iterative process flow :- It repeats one or more of the activities before proceeding to the next.



(c) Evolutionary process flow

- Evolutionary process flow : It executes the activities in a circular manner. Each circuit through five activities leads to a more complete version of software.



(d) Parallel process flow.

- Parallel process flow executes one or more activities in parallel with other activities.

Page No. :
Date : / /

Defining a framework.
Identifying a taskset.
Process patterns:- describes a process-related problem that is encountered during SE work, identifies environment in which problem has occured & suggest one or more solutions to the problem.

* Process Models:-

1) The waterfall model :- clearly defined requirements, whole product at once, delivery time of product is big, no risk assessment

- There are times when requirements for problem are well understood when workflows from communication through deployment in a reasonably linear fashion.
- This waterfall model also called as classic life cycle, suggests a systematic, sequential approach to software development that begins with customer specification of requirements & progresses through planning, modeling, construction & deployment.

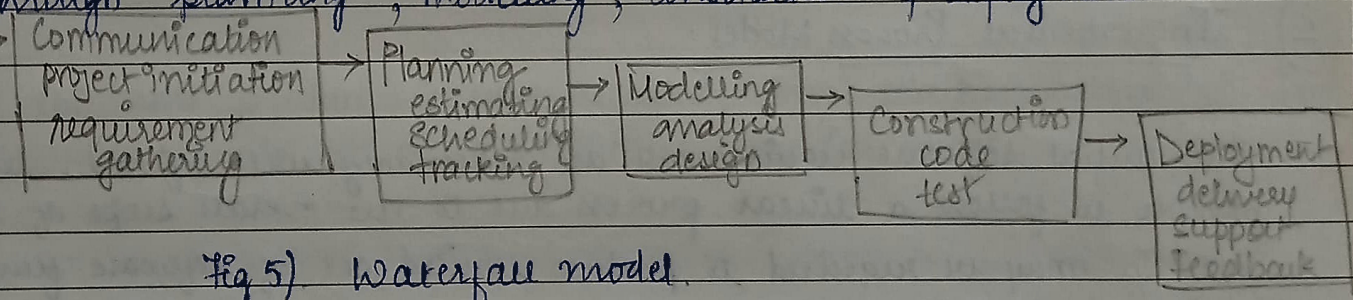


fig 5) Waterfall model.

V-model, a variation of waterfall model depicts relationship of quality assurance actions to the actions associated with communication, modelling & early construction activities.

As software team moves down left side of V, basic problem requirements are refined into progressively more detailed & technical representations of problem & solution.

After code generation, the team moves up rightside of V, performing series of tests, that validate each model created as team moved down the left side.

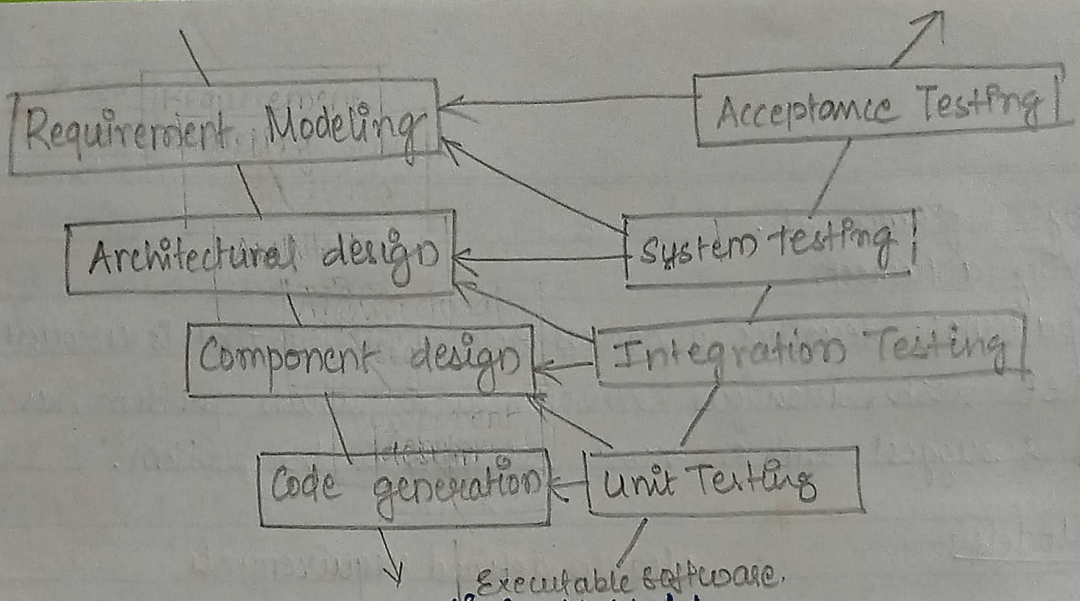


Fig 6. V-Model

Disadvantages of waterfall model:-

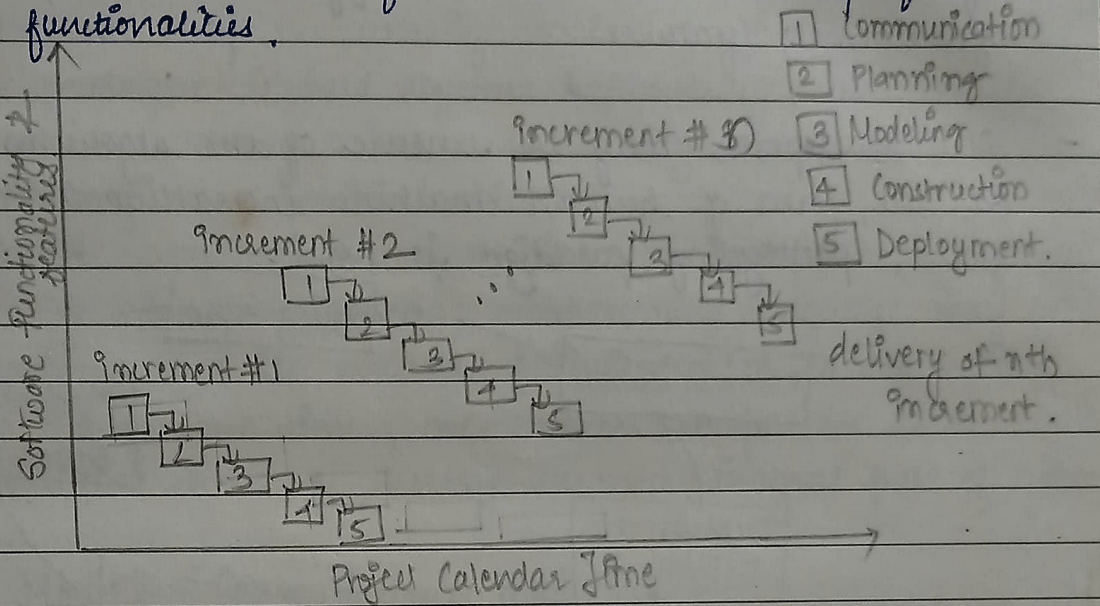
i) Real projects rarely follow sequential flow the model proposes.
ii) It is difficult for customers to state all requirements explicitly.
iii) The customer must have patience.
iv) Waterfall model may lead to blocking states in which some project team members must wait for other members of team to complete related task.

2) Incremental Process Model:

- The initial software requirements are reasonably well defined, but its not possible to follow a linear process due to the overall scope of project.
- Also it may be required to produce limited set of software functionality to users quickly & later on refine & expand on functionality in later software releases.
- Here, is when incremental process model is used.
- The incremental model combines elements of linear & parallel process flows. Each linear sequence produces deliverable 'increments' of software.
- Eg:- word processing software
  - → 1st increment basic file management, editing, document production capability
  - → 2nd increment spelling & grammar checking
  - → 3rd Increment advanced page layout capability.
- 1st increment is core product which addresses basic requirements but many supplementary features are undelivered
- This core product undergoes evaluation & a plan for next increment

is developed. The product developed through modification of core product better meets needs of customers with delivery of additional features & functionalities.

fig ④ incremental model.



- Incremental model is useful when limited staff is available for a project with a business deadline. This model delivers early product with limited core functionalities.

- If this core product is well received, then additional staff if required can be added to implement next increment.

- Incremental model helps manage risks. There may be a need of some hardware or software resources to develop a project, an early increment avoiding use of this increment enables partial functionality to be delivered to the end users without delay.

3) Evolutionary Process Models.

Need for evolutionary process models :-

- Business & product requirements often change overtime
- making straight line path to end product is unrealistic.
- tight market deadlines makes completion of comprehensive software product impossible.
- Limited version must be introduced to meet competitive & business pressure.

Prototyping Model :-

- Customer generally may define just a set of general objectives for software but does not identify detailed requirements for funtions & features.

- Also developers, may be unsure of the algorithms, Operating system or the form of human machine interaction needed. In such cases, prototyping paradigm is used.

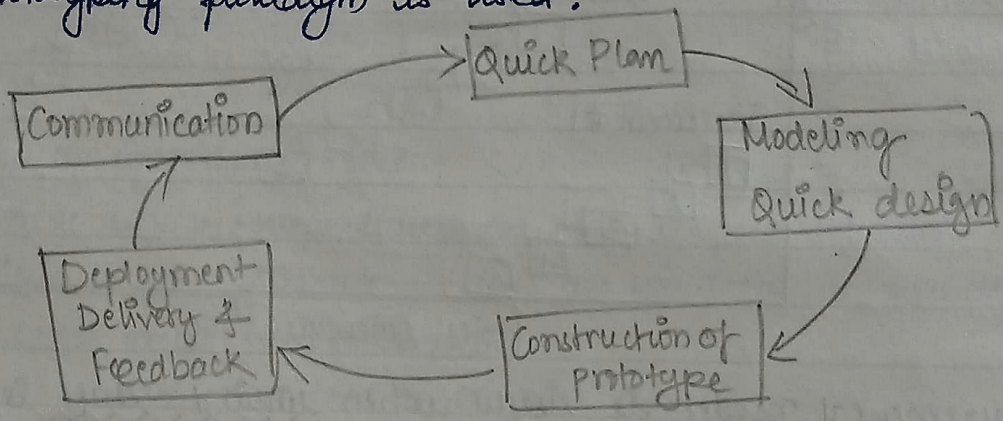


fig (8) Prototyping Model.

- Prototyping model begins with communication with all stakeholders to define overall objective of software,
- A prototyping iteration is planned and modeled quickly to represent those aspects of software that will be visible to endusers to construct an early prototype.
- The prototype is deployed & evaluated by stakeholders, who provide feedback that is used to further refine requirements.
- Iteration occurs as the prototype is tuned to satisfy the needs of various stakeholders, while at the same time enabling you to better understand what needs to be done.
- If a working prototype is used, it serves as a mechanism for identifying software requirements.
- Prototype can make use of existing program fragments or apply tools that enable working programs to be generated quickly.

Page No. :
Date : / /

Disadvantages of prototyping model:-

- Stakeholders see only the working version of software, unaware that the prototype is held together haphazardly, unaware in rush to get it working without considering overall software quality & long-term maintainability.
- Implementation compromises can be made in development of software prototype. An inappropriate os, programming language may be used simply because its available & implemented to demonstrate capability. After some time, one becomes comfortable with less-than-ideal choice & thereby becomes integral part of system.

* **Spiral Model** :- (Barry Boehm's model)

- It is an evolutionary model that couples iterative nature of prototyping with controlled systematic aspect of waterfall model.
- Spiral model is used to develop software in evolutionary releases. The early iteration develops a prototype & later iterations develops increasingly complete versions of system.

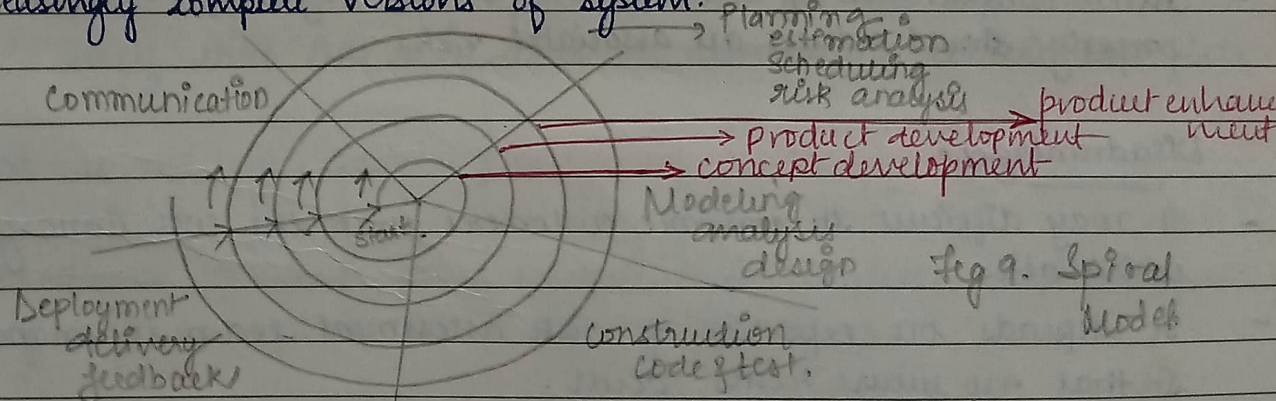


fig 9. Spiral Model

- In spiral model, one segment of spiral path is divided into number of framework activities that are implied by circuit around spiral in clockwise direction, beginning at centre.
- Risk is considered as each revolution is made.
- Anchor point milestones - a combination of work products & conditions are attained along the path of spiral.

- The first circuit around the spiral might result in development of a product specification; subsequent passes around spiral might be used to develop a prototype & then progressively more sophisticated versions of sfw.

- Each plan leads to adjustment in project plan. Cost & schedule are adjusted based on feedback derived from customer after delivery.

- Spiral model can be applied even after software is developed & delivered.
- The first circuit around the spiral might represent "concept development" that starts at the core of spiral & continues for multiple iterations until concept development is complete.
- If the concept is to be developed into actual product, the process proceeds outward on spiral & "new product development" commences.
- Later "product enhancements" can be carried out.
- Entry points can be used appropriately, like if change is initiated the product enhancement entry point can be used.

Advantages

- Systematic stepwise approach suggested by classic life cycle but incorporates it into iterative framework that reflects real world.
- direct consideration of technical risk at all stages of project & if applied properly should reduce all associated risk.

Disadvantages:

- It may difficult to convince customers that the evolutionary approach is controllable
- It depends on expertise of risk assessment team to manage major risks so that no future risks occur.

Page No. :
Date : / /

* Agile Model :-

- Agile software engineering combines a philosophy & set of development guidelines.
- This encourages customer satisfaction & early incremental delivery of software ; small, highly motivated project teams ; informal methods, minimal software engineering work products & overall development simplicity.
- The development guidelines stress delivery over analysis & design, & active & continuous communication between developers & customers

Steps of agile software development :

- communication
- planning
- modeling
- construction
- deployment.

→ These basic framework activities morph into minimal task set that pushes project team towards construction & delivery.

The most important work product here is the "software increment" itself that has to be done on appropriate commitment date.

* What is an agile process ?

Agile software process is characterised in a manner that addresses no. of key assumptions about software projects —

i) It is difficult to predict in advance which software requirements will persist & which will change. It is equally difficult to predict how customer priorities change as project proceeds.

ii) For many types of software, design & construction are interleaved. That is both activities should be performed so that design models are proven as they are created. It is difficult to predict how much design is necessary before constr-

ution is used to prove the design.
- Analysis, design, construction & testing are not as predictable as we might like.

- Given the three assumptions a process that can manage unpredictability ie agile process needs to be adaptable.

- Continual adaption without forward progress accomplishes little. Therefore, an agile team requires customer feedback to progress the software process with incremental adaptation.

- Software increments must be delivered in short time periods so that adaptations keep pace with change. This iterative approach enables the customer to evaluate the software increment regularly, provide necessary feedback to the software team, & influence process adaptations that are made to accommodate feedback.

* Agility Principles :-

- Agile principles are as follows:-

i) Our highest priority is to satisfy the customer through early & continuous delivery of valuable software.

ii) Welcome changing requirements, even late in development. Agile process harness change for customers competitive advantage.

iii) Deliver working software frequetly, from a couple of weeks to a couple of months, with a preference to shorter timescale.

iv) Business people & developers must work together daily throughout project

v) Build projects around motivated individuals. Give them the environment & support they need, & trust them to get job done.

vi) The most efficient & effective method of conveying information to and within a development team is face-to-face conversation.

vii) ~~The~~ working software is primary measure of progress.

viii) Agile process promote sustainable development. The sponsors, developers, & users should be able to maintain a constant pace indefinitely

ix) continuous attention to technical excellence & good design enhances agility
x) Simplicity - the art of maximizing amount of work not done - is essential
xi) The best architectures, requirements & designs emerge from self-organizing teams.
xii) At regular intervals, team reflects on how to become more effective, then tunes & adjusts its behavior accordingly.

* Human Factors:-

Characteristics of software team in agile process:

(i) Competence :- It encompasses innate talent, specific software related skills & overall knowledge of process that team has chosen to apply.

(ii) Common Focus :- Although members of agile team may perform different task & bring different skills to the project, all should be focussed on one goal - to deliver a working software increment within time promised.

(iii) Collaboration :- Software engineering is about assessing, analyzing & using information communicated to s/w team; creating information that will help all stakeholders understand the work of team; and building information that provides business value to customer.

(iv) Decision making ability :- The software team should be given autonomy to make decision for both technical & project issues

(v) Fuzzy problem-solving ability :- The team must accept the fact that the problem they are solving today may not be the problem that needs to be solved tomorrow. However, lessons learned from any problem-solving activity may be of benefit to the team leader in project

(vi) Mutual trust & respect :- A jelled team exhibits trust & respect that are necessary to make them "So strongly knit that the whole is greater than

the sum of the parts".

(vii) Self-organization

i) the agile team organizes itself for the work to be done

ii) the team organizes the process to best accomodate its local environment,

iii) the team organizes work schedule to best achieve delivery of software.

* EXTREME PROGRAMMING <XP> :-

* XP values :- There is a set of values established as a foundation for XP like

(i) communication :- To have effective communication between developers & customers ie other stakeholders XP emphasizes close, yet informal collaboration collaboration between customers & developers, establishment of effective ways to communicate & fetch continuous feedback without much documentation.

(ii) Simplicity :- Simplicity is achieved by developing design only for immediate needs, rather than considering future needs. If improvement is required in design it can be done later.

(iii) Feedback :- feedback is derived from three sources : the implemented software itself, the customer & other software team members.
The software provides agile team with unit test for each class developed to exercise each operation according to functionality
As the increment is delivered to the customer, the user stories & use cases implemented by the increment forms a basis for acceptance test. This feedback regarding degree to which function is attained forms a basis for feedback.

(iv) courage/discipline :- There is significant pressure to design for future requirements. Most software team fail, telling "design for tomorrow" will save time but the team should ideally follow "design for today" since requirements may dramatically change.

(v) **Respect**: The agile team inculcates respect amongst its members, between other stakeholders & team members for software itself. As they achieve successfully delivers software, the team develops growing respect for XP process.

* XP Process:-

- It is based on object oriented approach
- It encompasses set of rules and practices within the four framework activities of - planning, design, coding & testing

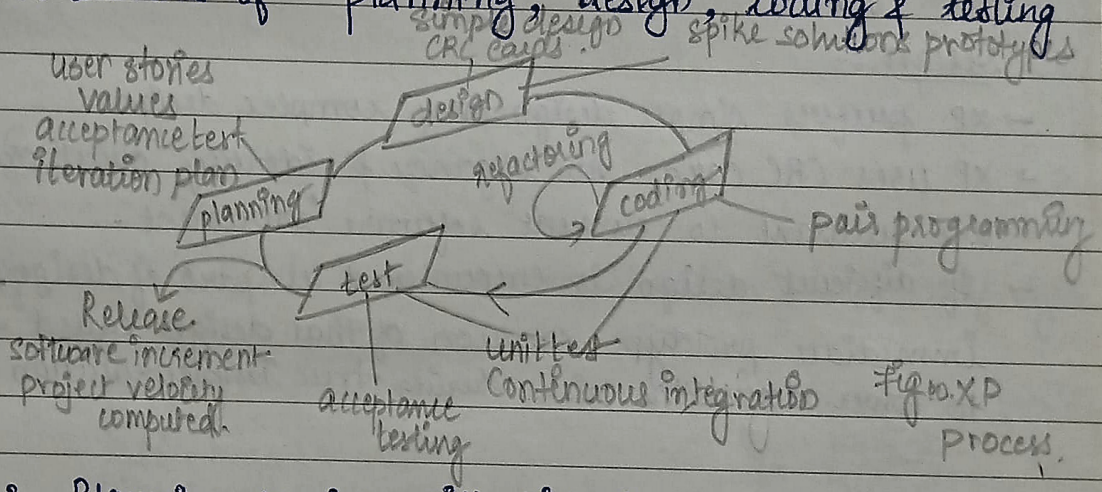


Fig 10. XP Process

- **Planning** :- Planning begins with listening a requirements gathering activities where technical team members get to know the features & functionality of software.

→ User stories are created from these features & functionalities of software
→ User assigns a priority value to these story based on overall business value of feature or function.
→ Members of XP team assess each story & assign a cost - measured in development weeks to it.
→ If the story needs more than two weeks then customer is asked to split the stories in smaller stories & assign value & cost again.
→ Continuous communicatn between customers & developers to plan next release.
→ After basic commitment is attained next release is planned in one of the three ways-
1) all stories will be implemented immediately

ii) the stories with highest value will be moved up in schedule & implemented first

iii) the riskiest stories will be moved up in schedule & implemented first.

Project velocity is the number of customers stories implemented during first release. Project velocity helps estimate –

1) delivery dates & schedule for subsequent releases.
2) determine whether an overcommitment has been made for all stories across entire development.
3) The XP team then reconsiders all remaining releases & modifies plan accordingly.

- **Design** :-

→ XP prefers simple design over complex design.
→ XP uses CRC cards to organize & identify object oriented classes that are relevant to current software increment.
→ If difficult design is encountered as part of design, XP recommends immediate prototype creation of that design called a spike solution. to lower down the risk during true implementation

- **Coding** :-

→ XP uses pair programming where two people work together at one computer to create a code for story.
→ This helps for real time problem solving & quality assurance
→ One person here might think about coding details of particular portion of design while others ensure coding standards are being followed.
→ 'continuous integration' strategy is used here to avoid compatibility & interfacing problems & provide early removal of errors.

- **Testing**:-

→ The unit tests are done even before coding commences which enables a framework for regression testing whenever code is modified
→ Also integration & validation testing is done on a regular basis which help in indication of progress as well as raise early alarms if certain things are wrong

Page No.:

Date: / /

Acceptance tests called customer tests are specified by customers & focus on overall system features & functionalities that are visible & reviewable by customers.

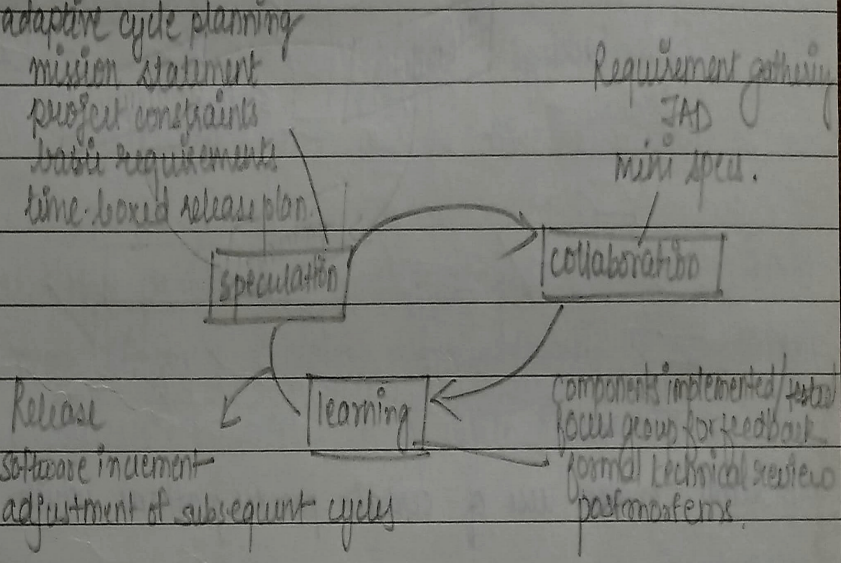
Issues of XP

- Requirement volatility
- Conflicting customer needs
- Requirements are expressed informally.
- Lack of formal design

* Adaptive Software Development (ASD) :-

The philosophy of this model is human collaboration & team self-organization.

Fig 11) ASD



- **Speculation** :- Here, the project is initiated & adaptive cycle planning is conducted. The planning here uses the customers mission statement, project constraints & basic requirements to define set of release cycles that will be required for project.

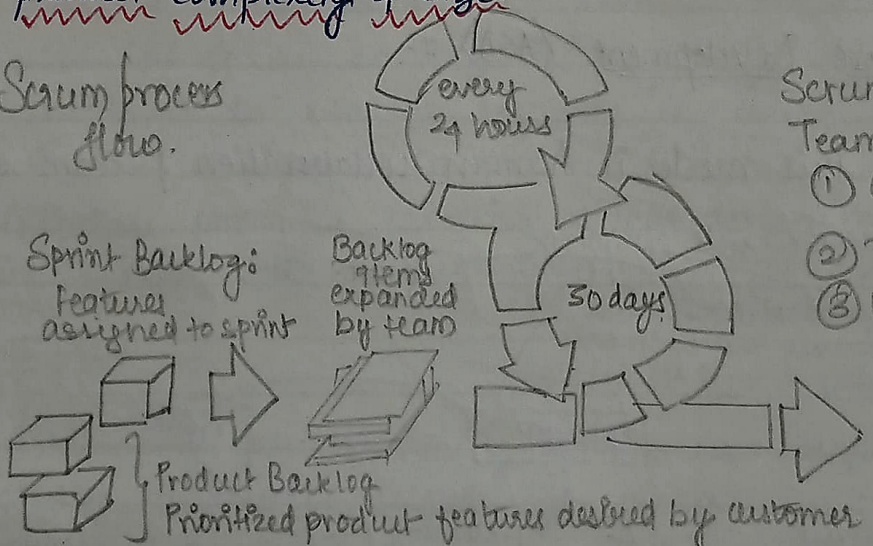
- **Collaboration** : It believes in a way that multiplies talent & creative output beyond discrete numbers. It encompasses communication & team work but also emphasizes on individual creativity.

- **Learning** :- Software developers overestimate their own understanding of tehnology, process & project this learning will help them to improve their level of real understanding. ASD teams learn through focus group technical reviews & project postmortems.

* Scrum :-

- Scrum principles guide development activities within a process that incorporates framework activities like requirement, analysis, design, evolution & delivery.
- Within each framework activity worktasks called sprints are formed.
- The number of sprints required for each framework activity varies depending on product complexity & size.

Fig. 12) Scrum process flow.



- Scrum makes use of certain process patterns which defines development actions like —

i) **Backlog** :- a prioritized list of project requirements or features that provide business value for customer.

Items can be added to backlog as per changing needs & prioritized by project manager.

ii) **Sprints** :- It consists of work units required to achieve a requirement defined in backlog that must fit in predefined time box.

Changes are not involved in sprint allowing a stable environment for team members.

Page No. :
Date : / /

iii) Scrum meetings : Short meetings of 15 mins held daily by team Certain questions asked & answered by all team members are
- What did you do since last meeting, obstacles encountered & what to accomplish by next meeting?

Scrum master leads these scrum meetings to uncover the potential problem as early as possible.

iv) Demos :- delivering the software increment to customer to demonstrate the developed functionality & evaluate it by customer

The product is just a timebox increment with agreed upon functions & not entire software with all functions

Development activities
Sprint tasks
No. of sprints required for each activity varies
patterns

Questions from University Papers:

* Dec 17 :- (KT Paper)

(i) What are the problems of prototyping model. — 5 mks.
(ii) With a neat diagram explain spiral model. — 10 mks.
(iii) Describe & discuss characteristics of agile requirements process. — 10 mks.
(iv) Write notes on - SCRUM — 10 mks.

* May 17 :-

(i) What is agility in context of software engineering? With suitable diagram explain Extreme Programming XP.

* Dec 2016

(i) Describe incremental & prototype model for software engg. with its merits & demerits. — 10 mks

(ii) Write notes on - Agile methodology — 10 mks

* May 2016

(i) What are agile process & its advantages? Explain any one agile process. — 10 mks

(ii)

* Dec. 2015

(i) Define Software Engineering. Explain the software process framework. (5)

(ii) What is agility in context of Software Engineering? Explain Extreme programming with suitable diagram.

* May 2015

(1) Write suitable application of different software models. (10)

(2) What is agile methodology? Explain it with principles used & give example of any one such Sfw model.

* Maturity Models :-

- A maturity model is applied within context of an SPI framework. The intent of maturity model is to provide an overall indication of "process maturity" exhibited by software organization.

- The quality indicates degree to which practitioners understand & apply the process.

- There are five levels of maturity :-

  - Level 5, Optimized :-
    The organization has quantitative feedback systems in place to identify process weaknesses & strengthen them proactively. Project teams analyze defects to determine their causes; software processes are evaluated & updated to prevent known types of defects from recurring.

  - Level 4, Managed :-
    Detailed software process & product quality metrics establish the quantitative evaluation foundation. Meaningful variations in process performance can be distinguished from random noise, & trends in process & product qualities can be predicted

  - Level 3, Defined :-
    Processes for management & engineering are documented, standardized & integrated into a standard software process for the organization. All projects use an approved, tailored version of organization's std software process for developing software.

  - Level 2, Repeatable :-
    Basic project mgmt processes are established to track cost,

Thank you

Whether I have a right to appeal to the Court of Appeal against the decision of the Court of Appeal in the case of [illegible] v. [illegible] [illegible] [illegible]